

INFORME DE PRUEBAS UNITARIAS - MOVENT

(AGENCIA DE VIAJES)

Fecha: 23 de abril de 2026

Proyecto: MOVENT - Sistema de Gestion de Viajes

Stack: Go/Gin + MySQL + Vue.js 3

RESUMEN EJECUTIVO

Estadisticas Generales

Metrica	Valor
Tests totales	56
Tests backend (Go)	20
Tests frontend (Vue/Vitest)	36
Tiempo ejecucion backend	0.576s
Tiempo ejecucion frontend	~2s
Tasa de exito	100%
Errores	0

Cobertura por Componente

Componente	Archivos	Tests	Estado
Services (Go)	2	20	CORRECTO

Components (Vue)	2	36	CORRECTO
TOTAL	4	56	100%

Estado del Build

```
Backend (Go 1.21) : ok  agencia-viajes/test/services 0.576s
Frontend (Vue 3)  : Test Files 2 passed | Tests 36 passed
```

1. BACKEND GO - SERVICES

Tecnologías: Go testing package, mocks manuales con structs, tests dirigidos por tabla (t.Run)

Archivos de test: 2

Tests totales: 20

1.1 LoginService (11 tests)

Archivo: test/services/login_service_test.go

Servicio: LoginService

Propósito: Autenticacion de usuarios con validacion de captcha y contraseña bcrypt

Modificaciones al código fuente

```
// ILoginRepository - interfaz para inyeccion de dependencias en tests
type ILoginRepository interface {
    ObtenerPorUsernameOCorreo(login string) (models.Usuario, error)
}

// Campo captchaVerifier permite inyectar funcion de verificacion en tests
type LoginService struct {
    repo          ILoginRepository
```

```

    captchaVerifier func(string) (bool, string)
}

// Constructor para pruebas unitarias
func NewLoginServiceConRepo(repo ILoginRepository,
    captchaV func(string) (bool, string)) *LoginService {
    return &LoginService{repo: repo, captchaVerifier: captchaV}
}

```

Tests implementados

#	Nombre del test	Comportamiento verificado
1	error si login esta vacio	Login vacio retorna ErrCamposVacios
2	error si contrasena esta vacia	Contrasena vacia retorna ErrCamposVacios
3	error si login es solo espacios	Espacios en blanco retorna ErrCamposVacios
4	error si captcha esta vacio	Sin captcha retorna ErrCaptchaAusente
5	error si captcha es solo espacios	Espacios en captcha retorna ErrCaptchaAusente
6	error si captcha es rechazado por google	Captcha invalido retorna ErrCaptchaInvalido
7	error si el repositorio falla en BD	Error de BD se propaga correctamente
8	error credenciales invalidas si usuario no existe en BD	ID=0 retorna ErrCredencialesInvalidas
9	error usuario deshabilitado si EstadoID distinto de 1	EstadoID!=1 retorna ErrUsuarioDeshabilitado
10	error credenciales invalidas si contrasena no coincide	Hash bcrypt no coincide retorna ErrCredencialesInvalidas
11	login exitoso retorna datos del usuario	Credenciales validas retorna LoginResponse con RolID

Ejemplo de test (tabla dirigida)

```

func TestLoginService_Login(t *testing.T) {
    tests := []struct {
        name      string
        req        dto.LoginRequest
        mock        mockLoginRepo
        captchaFn func(string) (bool, string)
        wantErr    bool
        wantErrIs  error
        wantRolID  int
    }{
        {
            name:      "login exitoso retorna datos del usuario",
            req:        buildLoginReq("ana_1", "password123", "tok"),
            mock:        mockLoginRepo{usuario: usuarioActivo()},
            captchaFn: captchaOk,
            wantErr:    false,
            wantRolID:  1,
        },
        // ... mas casos
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            svc := services.NewLoginServiceConRepo(&tt.mock, tt.captchaFn)
            resp, err := svc.Login(tt.req)
            // assertions...
        })
    }
}

```

1.2 UsuarioService (9 tests)

Archivo: test/services/usuario_service_test.go

Servicio: UsuarioService

Proposito: Validacion de campos unicos en el proceso de registro

Modificaciones al codigo fuente

```

// IUserarioRepository – interfaz con todas las operaciones necesarias
type IUserarioRepository interface {
    ExisteCorreo(correo string) (bool, error)
    ExistePasaporte(pasaporte string) (bool, error)
}

```

```

    ExisteUsername(username string) (bool, error)
    CrearUsuario(req dto.RegistroUsuarioRequest,
        ciudadID, rolID, estadoID int) (int, error)
    AsignarNacionalidades(usuarioID int,
        nacionalidadIDs []int) error
    ObtenerTodos() ([]dto.UsuarioResumen, error)
}

// Constructor para pruebas unitarias
func NewUsuarioServiceConRepo(repo IUsuarioRepository,
    ubicacionService *UbicacionService) *UsuarioService {
    return &UsuarioService{repo: repo,
        ubicacionService: ubicacionService}
}

```

Tests implementados

#	Nombre del test	Comportamiento verificado
1	todos los campos disponibles — sin conflictos	Los tres flags regresan false
2	correo ya registrado	Correo=true, Pasaporte=false, Username=false
3	pasaporte ya registrado	Pasaporte=true, resto false
4	username ya registrado	Username=true, resto false
5	correo y username ya registrados	Correo=true, Username=true, Pasaporte=false
6	los tres campos ya registrados	Los tres flags regresan true
7	error de BD al verificar correo	Error se propaga, no continua
8	error de BD al verificar pasaporte	Error se propaga, no continua
9	error de BD al verificar username	Error se propaga, no continua

Ejemplo de test

```

func TestUsuarioService_ValidarDatosUnicos(t *testing.T) {
    tests := []struct {
        name      string
        mock      mockUsuarioRepo
        wantErr   bool
        wantCorreo bool
    }
}

```

```

    wantPasaporte bool
    wantUsername bool
  }{
    {
      name:      "todos los campos disponibles – sin conflictos",
      mock:      mockUsuarioRepo{},
      wantCorreo: false,
      wantPasaporte: false,
      wantUsername: false,
    },
    {
      name:      "los tres campos ya registrados",
      mock:      mockUsuarioRepo{
        correoExiste: true,
        pasaporteExiste: true,
        usernameExiste: true,
      },
      wantCorreo: true,
      wantPasaporte: true,
      wantUsername: true,
    },
  },
  // ...
}

```

2. FRONTEND VUE

Tecnologías: Vitest 4, @testing-library/jest-dom, jsdom

Archivos de test: 2

Tests totales: 36

2.1 IniciarSesion.test.js (20 tests)

Vista: src/views/IniciarSesion.vue

Propósito: Validación de formulario, cálculo de destino por rol y persistencia en sessionStorage

Categorías de tests

Categoria	Tests	Descripcion
validateForm	7	Validacion de campos: login, password, captcha
calcularDestino	5	Ruta segun rol_id del usuario
buildSesion	4	Construccion del objeto de sesion
sessionStorage	4	Persistencia de datos post-login

Tests de validateForm (7 tests)

#	Test	Resultado esperado
1	Todos los campos completos	valid=true, sin errores
2	Login vacio	Error en campo login
3	Login con solo espacios	Error en campo login
4	Password vacio	Error en campo password
5	Captcha vacio	Error en campo captcha
6	Los tres campos vacios	Error en los tres campos
7	Login es correo electronico valido	valid=true

Tests de calcularDestino (5 tests)

#	Test	Resultado esperado
8	rol_id 1 (Cliente)	/principal
9	rol_id 2 (Admin)	/admin/dashboard
10	rol_id 3 (WebService)	/admin/webservice
11	rol_id desconocido	/principal (defecto)
12	rol_id undefined	/principal (defecto)

Tests de buildSesion y sessionStorage (8 tests)

#	Test	Resultado esperado
13	Cliente: isAdmin=false, isWS=false, rol=Registrado	Flags correctos
14	Admin: isAdmin=true, isWS=false, rol=Administrador	Flags correctos
15	WebService: isAdmin=false, isWS=true, rol=WebService	Flags correctos
16	Sesion incluye todos los campos requeridos	id, nombre, apellido, usuario, correo, rol, isAdmin, isWS
17	Cliente persiste en sessionStorage	nombre y isAdmin=false almacenados
18	Admin persiste en sessionStorage	isAdmin=true y rol=Administrador
19	sessionStorage vacio antes del login	getItem retorna null
20	Sesion admin serializable y recuperable	JSON.parse funciona correctamente

Ejemplo de test

```
describe('IniciarSesion - calcularDestino', () => {  
  it('rol_id 2 (Admin) → /admin/dashboard', () => {  
    expect(calcularDestino(2)).toBe('/admin/dashboard')  
  })  
  
  it('rol_id desconocido → /principal por defecto', () => {  
    expect(calcularDestino(99)).toBe('/principal')  
  })  
})
```

2.2 Encabezado.test.js (16 tests)

Componente: `src/components/Encabezado.vue`

Proposito: Carga de sesion desde sessionStorage y logica de navegacion condicional

Categorias de tests

Categoria	Tests	Descripcion
cargarSesion sin datos	2	sessionStorage vacio o JSON invalido
cargarSesion — cliente	3	Sesion con rol Registrado
cargarSesion — admin	4	Sesion con rol Administrador (3 variantes)
cargarSesion — WebService	4	Sesion con rol WebService (2 variantes + navbar)
nombreVisible	4	Computed basado en nombre/username/usuario

Tests detallados

#	Test	Resultado esperado
1	sessionStorage vacio	retorna null
2	JSON invalido	retorna null (catch silencioso)
3	Cliente: isAdmin=false	isAdmin es false
4	Cliente: isWS=false	isWS es false
5	Cliente: datos de sesion cargados	nombre correcto
6	Admin: isAdmin=true (flag directo)	isAdmin es true
7	Admin: isAdmin=true via rol_id=2	isAdmin es true
8	Admin: isAdmin=true via rol="Administrador"	isAdmin es true
9	Admin: isWS=false	isWS es false
10	WS: isWS=true (flag directo)	isWS es true

11	WS: isWS=true via rol_id=3	isWS es true
12	WS: isAdmin=false	isAdmin es false
13	WS: navbar muestra Panel WS (isWS && !isAdmin)	mostrarWS=true, mostrarAdmin=false
14	nombreVisible con nombre	retorna nombre
15	nombreVisible sin nombre, con usuario	retorna usuario
16	nombreVisible con sesion null	retorna 'Usuario'

Ejemplo de test

```
describe('Encabezado - cargarSesion usuario administrador', () => {
  it('isAdmin es true cuando rol_id es 2', () => {
    sessionStorage.clear()
    guardarSesion({ id: 5, nombre: 'Admin2', rol_id: 2 })
    const s = cargarSesion()
    expect(s.isAdmin).toBe(true)
  })

  it('isAdmin es true cuando rol === "Administrador"', () => {
    sessionStorage.clear()
    guardarSesion({ id: 6, nombre: 'Admin3',
      rol: 'Administrador', isAdmin: false })
    const s = cargarSesion()
    expect(s.isAdmin).toBe(true) // derivado del string de rol
  })
})
```

3. INFRAESTRUCTURA DE TESTS

Backend (Go)

Comando de ejecucion:

```
go test ./test/... -v
```

Resultado completo:

```
=== RUN   TestLoginService_Login
--- PASS: TestLoginService_Login/error_si_login_esta_vacio (0.00s)
--- PASS: TestLoginService_Login/error_si_contrasena_esta_vacia (0.00s)
--- PASS: TestLoginService_Login/error_si_login_es_solo_espacios (0.00s)
--- PASS: TestLoginService_Login/error_si_captcha_esta_vacio (0.00s)
--- PASS: TestLoginService_Login/error_si_captcha_es_solo_espacios (0.00s)
--- PASS: TestLoginService_Login/error_si_captcha_es_rechazado_por_google
(0.00s)
--- PASS: TestLoginService_Login/error_si_el_repositorio_falla_en_BD (0.00s)
--- PASS:
TestLoginService_Login/error_credenciales_invalidas_si_usuario_no_existe (0.00s)
--- PASS:
TestLoginService_Login/error_usuario_deshabilitado_si_EstadoID_distinto_de_1
(0.00s)
--- PASS:
TestLoginService_Login/error_credenciales_invalidas_si_contrasena_no_coincide
(0.00s)
--- PASS: TestLoginService_Login/login_exitoso_retorna_datos_del_usuario (0.21s)
=== RUN   TestUsuarioService_ValidarDatosUnicos
--- PASS: TestUsuarioService_ValidarDatosUnicos/todos_los_campos_disponibles
(0.00s)
--- PASS: TestUsuarioService_ValidarDatosUnicos/correo_ya_registrado (0.00s)
--- PASS: TestUsuarioService_ValidarDatosUnicos/pasaporte_ya_registrado (0.00s)
--- PASS: TestUsuarioService_ValidarDatosUnicos/username_ya_registrado (0.00s)
--- PASS: TestUsuarioService_ValidarDatosUnicos/correo_y_username_ya_registrados
(0.00s)
--- PASS: TestUsuarioService_ValidarDatosUnicos/los_tres_campos_ya_registrados
(0.00s)
--- PASS: TestUsuarioService_ValidarDatosUnicos/error_de_BD_al_verificar_correo
(0.00s)
--- PASS:
TestUsuarioService_ValidarDatosUnicos/error_de_BD_al_verificar_pasaporte (0.00s)
--- PASS:
TestUsuarioService_ValidarDatosUnicos/error_de_BD_al_verificar_username (0.00s)
ok  agencia-viajes/test/services 0.576s
```

Frontend (Vue/Vitest)

Configuracion vitest.config.js:

```
import { defineConfig } from 'vitest/config'
import vue from '@vitejs/plugin-vue'
```

```
export default defineConfig({
  plugins: [vue()],
  test: {
    environment: 'jsdom',
    globals: true,
    setupFiles: ['./src/tests/setup.js'],
    include: ['src/tests/**/*.test.js'],
  },
})
```

Scripts package.json:

```
{
  "scripts": {
    "test": "vitest run",
    "test:ui": "vitest --ui"
  }
}
```

Resultado completo:

```
RUN v4.1.5

✓ src/tests/IniciarSesion.test.js (20 tests) 21ms
✓ src/tests/Encabezado.test.js (16 tests) 6ms

Test Files 2 passed (2)
Tests 36 passed (36)
Duration ~2s
```

4. CONCLUSIONES

Logros alcanzados

- Cobertura inicial completa: 56 tests, 0 fallos
- Backend con mocks manuales: services independientes y testeables sin BD

- Captcha inyectable: helpers.VerificarCaptcha desacoplado via funcion
- Frontend con Vitest: testing moderno para Vue 3 Composition API
- Tiempo de ejecucion rapido: menos de 3 segundos en total

Metricas de calidad

Indicador	Valor	Estado
Tasa de exito	100%	Excelente
Tests backend	20	Completo
Tests frontend	36	Completo
Tiempo total	< 3s	Optimo
Deuda tecnica	Baja	Controlada

Areas cubiertas

Backend:

- Autenticacion con captcha (11 casos)
- Validacion de campos unicos en registro (9 casos)
- Manejo de errores de BD en todos los flujos

Frontend:

- Validacion de formularios (login, password, captcha)
- Calculo de destino post-login por rol
- Construccion del objeto de sesion
- Persistencia y recuperacion en sessionStorage
- Navegacion condicional por rol (Admin / WebService / Cliente)

ANEXO: COMANDOS UTILES

Backend

```
go test ./test/... -v
go test ./test/services/... -v
go test ./test/... -cover
```

Frontend

```
npm test
npm run test:ui
```

FIN DEL INFORME